

CODE SECURITY AND DATA PRIVACY

This section outlines the technical strategies and defense mechanisms implemented to ensure the integrity of the StockWise Smart Inventory System and the confidentiality of its users' sensitive data.

Code Security

To protect the application from common vulnerabilities and unauthorized access, a multi-layered security approach has been adopted:

A. Authentication and Authorization

- **JWT (JSON Web Token) Integration:** The system utilizes a stateless authentication mechanism. Upon login, users receive a signed JWT. This token must be included in the header of every subsequent request. We have implemented a dual-token system: **Access Tokens** (1-hour lifespan) and **Refresh Tokens** (7-day lifespan) to balance security and user experience.
- **Role-Based Access Control (RBAC):** Access is strictly governed by user roles.
 - **ROLE_ADMIN:** Full access to product management, category CRUD, user administration, and financial reporting.
 - **ROLE_STAFF:** Limited access to real-time stock monitoring and basic stock updates.
- **Method-Level Security:** Using Spring Security's `@PreAuthorize` annotations, authorization checks are performed at the code level before any service method is executed, preventing "Insecure Direct Object Reference" (IDOR) attacks.

B. Defensive Coding Practices

- **Input Validation & Sanitization:** All user inputs (product names, stock quantities, prices) are validated on both the **Frontend (React/Formik)** and **Backend (Jakarta Bean Validation)**. This prevents malformed data from entering the system and mitigates **Cross-Site Scripting (XSS)** risks.
- **SQL Injection Mitigation:** By utilizing **Spring Data JPA** and **Hibernate**, the system uses prepared statements and parameterized queries by default. This ensures that user input is never treated as executable SQL code.
- **Secure Dependency Management:** We monitor Maven and npm dependencies for known vulnerabilities (CVEs) to ensure the third-party libraries used in the tech stack are up to date.

Data Privacy

StockWise prioritizes the privacy of business-critical information, including inventory costs, payroll data, and user credentials.

A. Data Encryption

- **Hashing of Credentials:** User passwords are never stored in plain text. We utilize the **BCrypt** adaptive hashing function with a strong work factor and unique salts for every user to protect against rainbow table and brute-force attacks.
- **Encryption in Transit:** In the production environment, all communication between the React frontend and Spring Boot backend is encrypted via **TLS/SSL (HTTPS)**, ensuring that data cannot be intercepted by unauthorized parties.

B. Sensitive Information Management

- **Environment Variables:** Sensitive configuration data—such as database credentials, JWT secret keys, and SMTP server details—are never hardcoded. These are managed through **System Environment Variables**, ensuring they stay out of the version control system (GitHub).
- **Payroll Privacy:** In the newly proposed payroll module, data visibility is restricted using a "Least Privilege" model. Each team member can only view their own calculated salary, while only the Project Manager (Admin) has access to the global payroll matrix.

C. Secure Logging and Error Handling

- **Data Masking:** Sensitive information is filtered out of application logs. Passwords and personal identifiers are masked (e.g., ****) to prevent accidental exposure in log management tools.
- **Generic Error Messaging:** The API is configured to return generic error messages to the client. Detailed system traces (stack traces) are hidden from the user to prevent "Information Leakage" that could be used by attackers to map the system architecture.

Audit and Compliance

- **JPA Auditing:** The system tracks metadata for every entity. Using `@CreatedDate` and `@LastModifiedBy`, we maintain an audit trail of who changed stock levels and when, ensuring accountability for inventory discrepancies.
- **Session Management:** To prevent session hijacking, we implement secure cookie attributes and strict token validation checks on the backend for every RESTful call.

StockWise Security Compliance Checklist (OWASP Top 10 Mapping)

OWASP Category	Potential Vulnerability	StockWise Mitigation Strategy (Your Implementation)
A01: Broken Access Control	Users accessing Admin reports or deleting products.	RBAC & Method Security: Implementation of <code>ROLE_ADMIN</code> vs <code>ROLE_STAFF</code> using Spring Security <code>@PreAuthorize</code> on REST endpoints.
A02: Cryptographic Failures	Storing passwords in plain text or using weak hashes.	BCrypt Hashing: All user credentials are salted and hashed using the BCrypt algorithm before being persisted to PostgreSQL.
A03: Injection	Attackers executing malicious SQL through search bars.	JPA & Parameterized Queries: Use of Spring Data JPA repositories ensures that input is never concatenated directly into SQL strings.
A04: Insecure Design	Lack of protection for sensitive payroll or stock data.	Layered Architecture: Clear separation between Controller, Service, and Repository layers with dedicated Security Filters.
A05: Security Misconfiguration	Exposing database passwords in <code>application.properties</code> .	Environment Variables: Critical secrets (<code>DB_URL</code> , <code>JWT_SECRET</code>) are injected via OS environment variables, not hardcoded in the repo.
A07: Identification & Auth Failures	Session hijacking or weak session management.	JWT (Stateless) Auth: Secure token-based authentication with 1-hour expiration and 7-day Refresh Token rotation.
A09: Security Logging & Monitoring	No record of who changed critical inventory levels.	JPA Auditing: Usage of <code>@CreatedBy</code> and <code>@LastModifiedDate</code> to track every change in the <code>PRODUCTS</code> and <code>ALERTS</code> tables.